Actualizar y probar todo el sistema

Guía paso a paso para el equipo del Portal Administrativo · Club Atlético Camioneros de Córdoba · octubre de 2026

---

0. Qué vas a lograr

Al terminar vas a tener, **en tu máquina**, los dos portales funcionando juntos con una base de datos de prueba:

- El **Portal Administrativo** (nuestro): API y pantallas, con el cambio a la base unificada, el login único y la inscripción por género.
- El **Portal Deportivo** (del otro equipo), con los cambios que le preparamos, para probar el recorrido completo: alta de jugador, cuota, cobro, QR, baja.

> **No se toca nada de lo que ya tenés.** Todo corre sobre copias en una carpeta aparte y sobre una base nueva de prueba (`ClubCamionerosUnificadaV2_Prueba`). Tu base actual, tu `appsettings.Development.json` y tu código quedan como están.

1. Antes de empezar

1.1 Qué tiene que estar instalado

☐ Git para Windows.

☐ .NET SDK 9.

☐ Node.js 20.19 o superior.

☐ Google Chrome.

☐ SQL Server (Express alcanza) con la herramienta `sqlcmd`, y que su usuario de Windows pueda crear bases (`localhost` con autenticación de Windows).

☐ PowerShell (viene con Windows).

1.2 Qué repositorios hacen falta

Repo	Para qué	Cómo se usa
Sistema-Administrativo---CACC (el nuestro)	Nuestro código	Se actualiza a la rama `feat/base-unificada`
cacc-unificacion	Scripts que arman todo el entorno y lo prueban	Se clona al lado, no se modifica
App-Club-Atletico-Camioneros (el del otro equipo)	Su código	Se clona **solo para leerlo**; los scripts le aplican los cambios sobre una copia

Si alguno no les abre en GitHub (repo privado), pidan acceso a la persona que les pasó esta guía.

2. Paso 1 · Poner nuestro repo al día

La rama con todos los cambios es `feat/base-unificada`. Abran PowerShell **dentro de la carpeta de nuestro repo**:

```
cd "C:\ruta\a\Sistema Administrativo - CACC"
git status
```

Si `git status` dice que hay cambios sin guardar

Antes de seguir, guárdenlos aparte (no se pierden):

```
git stash push -u -m "mis cambios antes de pasar a feat/base-unificada"
```

Si tienen **commits propios** que todavía no subieron, avisen antes de seguir: hay que ver cómo combinarlos con esta rama.

Traer y cambiar de rama

```
git fetch origin
git checkout feat/base-unificada
git pull
git log -1 --oneline
```

Debe mostrar el último commit de la rama (el mensaje empieza con `feat:`). Para confirmar que están en la rama correcta: `git branch --show-current` debe decir `feat/base-unificada`.

3. Paso 2 · Bajar los otros dos repos

```
cd C:\ruta\donde\quieran
git clone https://github.com/Atzur1/cacc-unificacion.git
git clone https://github.com/NejoCHZ/App-Club-Atletico-Camioneros.git
```

Quedan tres carpetas hermanas. No hace falta cambiar de rama en esos dos: el del otro equipo se usa en `main`.

4. Paso 3 · Armar el entorno de prueba (una sola vez)

Este script hace todo: copia el código, aplica los cambios al sistema del otro equipo, crea la base de prueba con sus catálogos y 3 cuentas, escribe la configuración con una clave JWT compartida y compila las dos APIs.

```
cd C:\ruta\a\cacc-unificacion
.\scripts\armar.ps1 -RepoNuestro "C:\ruta\a\Sistema Administrativo - CACC" -RepoEllos "C:\ruta
\a\App-Club-Atletico-Camioneros" -Recrear
```

- Si PowerShell bloquea el script: `Set-ExecutionPolicy -Scope Process Bypass` y repetirlo.
- Al empezar muestra "Versiones esperadas...": es solo un aviso informativo, se puede ignorar. Sí tiene que decir **OK ellos-definitivo.patch** más adelante.
- `-Recrear` borra y vuelve a crear **solamente** la base de prueba `ClubCamionerosUnificadaV2_Prueba`. Es seguro repetirlo.
- Si el servidor SQL no es `localhost`: agregar `-Servidor "localhost\SQLEXPRESS"`.

Resultado esperado: termina con "Compilación correcta" dos veces y sin ERROR en rojo.

5. Paso 4 · Dos formas de probar

A) Prueba automática (10 minutos)

```
.\scripts\probar.ps1
.\scripts\detener.ps1
```

Levanta las dos APIs y corre 88 verificaciones contra la base real. Debe terminar con: `RESUMEN: 88 PASAN | 0 FALLAN | 0 GAP`. El detalle queda en `trabajo\resultado.txt`.

Ojo: `probar.ps1` deja datos de prueba en la base. Si después quieren probar a mano, vuelvan a correr el Paso 3 (`armar.ps1 … -Recrear`) para empezar de cero.

B) Prueba manual en el navegador (la que más interesa)

```
.\scripts\manual.ps1 -RepoNuestro "C:\ruta\a\Sistema Administrativo - CACC" -RepoEllos "C:\ruta
\a\App-Club-Atletico-Camioneros"
```

- La primera vez tarda varios minutos (instala las dependencias de los dos frontends).
- Abre dos ventanas de PowerShell (los frontends). **Esperen** a que cada una diga `Local: http://localhost:...`.
- Para frenar todo: `.\scripts\manual.ps1 -Detener`

Qué	Dirección
Portal Administrativo (empezar siempre acá)	**http://localhost:4200**
Portal Deportivo	http://localhost:4300 (no se entra directo: se llega desde el Administrativo)
API Administrativo / API Deportivo	http://localhost:5118 / http://localhost:5191

Importante: la API que corre es una **copia** hecha en el Paso 3. Si cambian el código de la API o vuelven a hacer `git pull`, hay que repetir el Paso 3 para que la copia se actualice. Los frontends sí se leen directo de sus carpetas.

Cuentas de prueba

Usuario = el DNI. Contraseña de todas: `Camioneros.Test.2026!`

DNI	Quién es	Adónde entra
20111222	SuperAdmin	Elección de portales (ve los dos)
30000003	Médica (solo Deportivo)	Directo al Deportivo
30000004	Director Técnico, categoría 3 (solo Deportivo)	Directo al Deportivo
30000002	Ana, candidata a Administradora	No tiene acceso hasta que el SuperAdmin la habilite (ver 6.6)

6. Paso 5 · Qué probar a mano

La base arranca **vacía**: no hay jugadores ni aranceles. Por eso el orden importa: primero aranceles, después alta de jugadores. Tilden cada punto.

6.1 Login y portales

☐ Entrar a `localhost:4200` con `20111222`: aparece la elección de portales (Administrativo, Deportivo, Personas).

☐ Clave incorrecta: rechaza con mensaje claro.

☐ Entrar con la Médica (`30000003`): va **directo** al Deportivo, sin otro login.

☐ Abrir `localhost:4300` sin sesión: no muestra login, te manda al del Administrativo.

☐ En el Deportivo, "Portales": vuelve a la elección de portales **nuestra**, con la sesión abierta.

6.2 Aranceles (Portal Administrativo → Actualización de aranceles)

☐ Programar el arancel de cuota **Masculino** (por ejemplo $85.000, vigente desde hoy) y el **Femenino** (por ejemplo $70.000).

☐ Programar la **inscripción Masculino** (por ejemplo $70.000).

☐ La inscripción **Femenino** se puede programar en **$0**: se muestra como "No se cobra".

☐ No deja repetir un arancel del mismo género el mismo día.

☐ El monto mínimo de cuotas sigue exigiendo más de $0 (solo la inscripción admite $0).

6.3 Alta de jugadores (tarjeta "Personas", que abre el alta del Deportivo)

☐ Desde la elección de portales, "Personas" abre el alta de jugadores y staff del Deportivo.

☐ Dar de alta un jugador **sin género** o **sin categoría**: no deja guardar.

☐ Dar de alta un varón: cobra inscripción masculina. Dar de alta una mujer: cobra la femenina (o nada si está en $0).

☐ El jugador aparece en el Administrativo (buscador de jugadores, Cuotas y pagos).

6.4 Cuotas, deuda y cobro

El generador de cuotas corre al iniciar la API y cada 6 horas, y **emite la cuota de un mes recién desde el día 10 del mes siguiente** (regla del club). Para ver morosidad sin esperar, carguen cuotas vencidas por SQL. Reemplacen `ID` por el id del jugador (`SELECT PK_id_jugador FROM JUGADORES`) y las fechas por el primer día de dos meses anteriores ya vencidos:

```
sqlcmd -S localhost -E -C -I -d ClubCamionerosUnificadaV2_Prueba -Q "INSERT PAGOS(FK_id_jugado
r,monto_base,monto_final,fecha_vencimiento,estado,concepto) VALUES (ID,85000,85000,'2026-08-0
1',0,'Cuota'),(ID,85000,85000,'2026-09-01',0,'Cuota')"
```

☐ Deudas y morosidad: el jugador aparece como moroso con 2 cuotas vencidas.

☐ Cobrar la cuota más vieja (Cuotas y pagos): queda 1 vencida.

☐ Aplicar una beca o descuento (Becados y descuentos) y ver que cambia el saldo.

☐ El dashboard (Resumen general) muestra los números esperados.

6.5 QR y estado deportivo

En el Deportivo (como SuperAdmin) abrir la pantalla del QR e ingresar el DNI a mano:

☐ Jugador al día: **verde**, registra asistencia. Escanear de nuevo el mismo día: "ya registrada".

☐ Con 1 cuota vencida: **amarillo**, deja pasar.

☐ Con 2 o más cuotas vencidas: **rojo**, no registra asistencia.

☐ **Apagar la API del Administrativo** (cerrar su proceso) y escanear: el QR dice que no pudo consultar la deuda y **no deja pasar**.

6.6 Habilitar una cuenta (SuperAdmin) y roles

☐ Como SuperAdmin, habilitar a Ana (`30000002`) como Administradora. El sistema no manda correo en pruebas: el **enlace de activación** aparece en el archivo `cacc-unificacion\trabajo\log_nuestro.txt` .

☐ Ana crea su contraseña con ese enlace. Después entra por DNI y va **directo al Portal Administrativo** (no ve la elección de portales).

☐ Ana **no** ve Auditoría y **no** puede programar aranceles de inscripción.

☐ Ana no puede entrar al Deportivo.

6.7 Bajas y cierre de sesión

☐ Dar de baja un jugador: desaparece de los listados del Deportivo y el QR lo rechaza.

☐ Reincorporarlo: vuelve con su historial de pagos.

☐ Cerrar sesión en el Administrativo y, en otra pestaña con el Deportivo abierto, hacer cualquier acción: pide volver a ingresar.

7. Cómo avisar si algo falla

Para que podamos arreglarlo rápido, manden por cada problema:

1. **Qué hicieron**, paso a paso (el número de punto de esta guía sirve: "6.4, tercer punto").
2. **Qué esperaban** y **qué pasó**.
3. **Una captura** de la pantalla.
4. Las últimas líneas de los registros: `cacc-unificacion\trabajo\log_nuestro.txt` y `log_nuestro_err.txt` (API Administrativo), `log_ellos.txt` (API Deportivo). Antes de mandarlos, **revisen que no tengan contraseñas ni claves**.

8. Problemas frecuentes

Síntoma	Qué hacer
`armar.ps1` dice que no puede conectar o crear la base	Revisar que SQL Server esté corriendo y que `sqlcmd -S localhost -E -C -Q "SELECT 1"` funcione. Si la instancia tiene otro nombre, usar `-Servidor`.
"El parche no aplica"	El repo del otro equipo no está en `main` actualizado, o tiene cambios locales. Hacer `git checkout main` y `git pull` en esa carpeta.
Los puertos 5118, 5191, 4200 o 4300 están ocupados	Correr `.\scripts\manual.ps1 -Detener` y volver a intentar; si sigue, cerrar el programa que los usa.
Un frontend dice "No pasa nada" al abrirlo	Esperar a que la ventana de PowerShell correspondiente muestre `Local: http://localhost:...` (la primera compilación demora).
Login: "credenciales incorrectas"	Usuario = DNI sin puntos; contraseña `Camioneros.Test.2026!` con mayúsculas y signo de cierre.
Todo da 401 de golpe	Se cerró la sesión o venció el token. Volver a entrar por `localhost:4200`.
Quiero empezar de cero	`.\scripts\manual.ps1 -Detener`, después repetir el Paso 3 y el Paso 4B.

9. Qué NO hacer

- No subir a git la carpeta `cacc-unificacion\trabajo` (ya está ignorada) ni ningún `appsettings.Development.json`.
- No usar estas cuentas de prueba ni esta base fuera de su máquina.
- No cambiar código dentro de `trabajo\`: es una copia que se pisa cada vez que corren `armar.ps1`.

Más detalle técnico: `cacc-unificacion\docs\parche-deportivo.md` y `cacc-unificacion\docs\hallazgos-otro-equipo.md`.